

Mecanum wheel control in the simulator

Lab course experiment 1 — Intelligent Robotics

Supervision: Robotics Lab

Version 0.1 — Winter semester

You write the *control of the four Mecanum wheels* of a mobile robot. For that you convert a commanded body velocity into four wheel speeds in rad/s and publish them on a ROS topic. A small 2D simulator (Pygame, about 1400 lines of Python) drives your code in one of three environments and returns odometry, LIDAR and a global position. The simulator is deliberately tiny: you can read the physics in half an hour and are then able to change it for your own questions.

Contents

1. Experiment goal and prelab questions	2
1.1. Goal	2
1.2. Prelab questions (work them out before the lab date)	2
2. The Mecanum principle	3
2.1. Roller layout	3
2.2. Velocity decomposition at one wheel	3
2.3. Forward kinematics as a pseudo-inverse	4
3. The simulator	4
3.1. Architecture	4
3.2. Topics and types	4
3.3. What is noisy, what is exact	4
3.4. pass-through mode and wheels mode	5
3.5. A second drive train: the steering car	5
3.6. The radio link: commands that can get lost	6
4. Getting started	7
4.1. Installation	7
4.2. Trying it out	7
4.3. Creating a robot through a service	8

4.4. Pitfalls	8
5. Tasks	9
6. Submission and grading	10
7. Outlook	11
A. Topic, type and API overview	11
B. Formula sheet	12
C. Solutions to the prelab questions	12

1. Experiment goal and prelab questions

1.1. Goal

After the experiment you can

- derive the kinematics of a Mecanum drivetrain from the roller layout and justify the signs (instead of copying them),
- explain the difference between inverse and forward kinematics and their pseudo-inverse solution,
- write a ROS 2 node that reads topics (`odom`, `scan`, `gps`) and publishes (`cmd_vel`, `wheel_speeds`),
- measure the drift of a dead-reckoning odometry and put it in perspective.

Work is solo, one person each: one robot name per participant, one node per robot. The grader measures exclusively over the topics — how you implement it is up to you.

1.2. Prelab questions (work them out before the lab date)

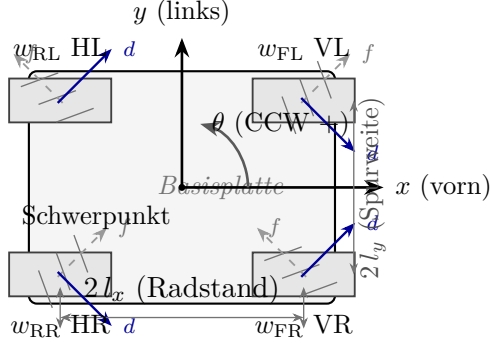
Use the simulator values from table 2: $l_x = 0.14$ m, $l_y = 0.13$ m, $r = 0.05$ m, $a = l_x + l_y = 0.27$ m, maximum wheel speed $w_{\max} = 12$ rad/s. Solutions: appendix C.

1. **Forward.** How fast does each of the four wheels turn at $v_x = 0.4$ m/s, $v_y = \omega = 0$?
2. **Sideways.** Wanted: $v_y = +0.25$ m/s (to the left). Compute the four wheel speeds and explain why two wheels are negative although the robot does not turn.
3. **Turning on the spot.** For $\omega = +1.2$ rad/s (counter-clockwise): what signs do FL and RL have — and why the same?
4. **Diagonal drive.** $v_x = v_y = 0.3$ m/s. One wheel stands still. Which one? What does that mean for the highest achievable diagonal speed if all wheels are limited by $w_{\max}$?
5. **Working backwards.** The wheels measure $[w_{\text{FL}}, w_{\text{FR}}, w_{\text{RL}}, w_{\text{RR}}] = [8, 0, 0, 8]$ rad/s. Determine v_x , v_y , ω and state the driving direction and its magnitude.

2. The Mecanum principle

2.1. Roller layout

A Mecanum wheel is an ordinary wheel whose tread consists of rollers mounted at an angle (45°). It can (i) roll, driven like an ordinary wheel, in its drive direction d and (ii) glide freely perpendicular to it, along f . In the top view the four drive directions point forward-inward; an X appears — the *X layout* (figure 1).



Rand-und-Vorn-Regel: x nach vorn, y nach *links*, θ gegen den Uhrzeigersinn
 Rollachsen-Pfeile d ergeben in der Draufsicht ein X (*X-Anordnung*).
 f = Freigleit-Richtung (dreht das Fahrzeug auf der Stelle, bleibt unvermittelt).
 Radreihenfolge in allen Themen: $[w_{FL}, w_{FR}, w_{RL}, w_{RR}] = [VL, VR, HL, HR]$

Figure 1: Mecanum drivetrain from above. Blue arrows d = driven direction, dashed arrows f = free-gliding direction, hatched strips = rollers. Axes and signs are exactly those of the simulator.

2.2. Velocity decomposition at one wheel

The centre of mass has the body velocity $\xi = (v_x, v_y, \omega)^T$; wheel i sits at the position $\mathbf{p}_i = (\pm l_x, \pm l_y)$. Its contact point moves with

$$\mathbf{v}_i = \begin{pmatrix} v_x \\ v_y \end{pmatrix} + \omega \begin{pmatrix} -p_{i,y} \\ p_{i,x} \end{pmatrix}, \quad \text{e.g. for FL: } \mathbf{v}_{FL} = \begin{pmatrix} v_x - l_y \omega \\ v_y + l_x \omega \end{pmatrix}. \quad (1)$$

The wheel delivers speed only in its drive direction; in the free-gliding direction it transfers nothing. With the drive directions from fig. 1, $d_{FL} = (1, -1)$, $d_{FR} = (1, 1)$, $d_{RL} = (1, 1)$, $d_{RR} = (1, -1)$ (projection without normalization — the factor $\sqrt{2}$ sits in the effective wheel radius calibration), $r w_i = \mathbf{v}_i \cdot d_i$ follows:

$$r w_{FL} = (v_x - l_y \omega) - (v_y + l_x \omega) = v_x - v_y - a \omega \quad (2)$$

$$r w_{FR} = (v_x + l_y \omega) + (v_y - l_x \omega) = v_x + v_y + a \omega \quad (3)$$

$$r w_{RL} = (v_x - l_y \omega) + (v_y + l_x \omega) = v_x + v_y - a \omega \quad (4)$$

$$r w_{RR} = (v_x + l_y \omega) - (v_y - l_x \omega) = v_x - v_y + a \omega \quad (5)$$

with the yaw lever arm $a = l_x + l_y$. That is the **inverse kinematics** — your task in T1. As a matrix (wheel order [FL, FR, RL, RR]):

$$\mathbf{w} = J \boldsymbol{\xi}, \quad J = \frac{1}{r} \begin{pmatrix} 1 & -1 & -a \\ 1 & 1 & a \\ 1 & 1 & -a \\ 1 & -1 & a \end{pmatrix}. \quad (6)$$

2.3. Forward kinematics as a pseudo-inverse

Wanted is $\boldsymbol{\xi}$ from measured wheel speeds, so $\mathbf{w} = J\boldsymbol{\xi}$ with four equations and three unknowns. Since J has column rank 3, the Moore-Penrose solution ⁺ is the pseudo-inverse $J^+ = (J^T J)^{-1} J^T$. It is $J^T J = \frac{1}{r^2} \text{diag}(4, 4, 4a^2)$ — the three degrees of freedom are *decoupled*, and this results in

$$J^+ = \frac{r}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{a} & \frac{1}{a} & -\frac{1}{a} & \frac{1}{a} \end{pmatrix} \Rightarrow \begin{aligned} v_x &= \frac{r}{4} (w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} + w_{\text{RR}}) \\ v_y &= \frac{r}{4} (-w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} - w_{\text{RR}}) \\ \omega &= \frac{r}{4a} (-w_{\text{FL}} + w_{\text{FR}} - w_{\text{RL}} + w_{\text{RR}}) \end{aligned} \quad (7)$$

The simulator uses eq. (7) for the odometry: what the wheels actually do decides where the robot ends up — not what you wanted. That is why a wrong sign in T1 shows up at once as a wrong driving direction.

Sign box (mnemonic).

x forward, y to the **left**, θ counter-clockwise positive. Wheel order in *all* topics: [FL, FR, RL, RR] = [VL, VR, HL, HR]. Wheel speed in rad/s, body velocity in m/s and rad/s respectively.

3. The simulator

3.1. Architecture

The simulator exists in two flavours, on the same code: *with* ROS 2 (`ros_bridge.py` → real DDS bus) and *without* ROS (`stub.py` → bus in the same process). For you the student code is identical.

3.2. Topics and types

Frames: `map` (world), `odom` (odometry origin), `base_link` (robot), `laser`. Robot names carry their topics as a namespace.

3.3. What is noisy, what is exact

Exact (deterministic, no noise) are only the *truth* in `physics.py` and — if switched on — the topic `<robot>/truth`. Four wheel speeds give a unique (v_x, v_y, ω) by eq. (7) — with one deliberate exception: against an obstacle the body stands still while the wheels keep their speed (`robot.slip`, default 1). Odometry integrates wheel speeds, so it then counts metres that were never driven. Drive into a wall with the keyboard

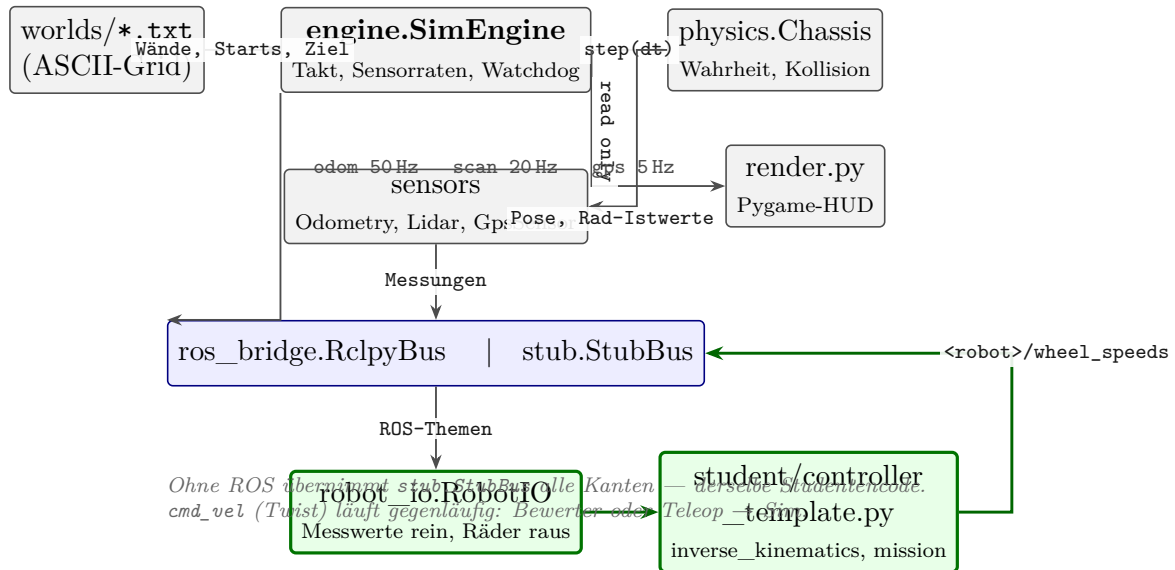


Figure 2: Data flow. The closed control loop runs over `wheel_speeds`: only your four wheel speeds determine the pose. The renderer is purely read-only.

(the up arrow) and watch `odom` in the readout run ahead of the picture; with `-set robot.slip=0` the wheels stand still and the odometry is honest again.

3.4. pass-through mode and wheels mode

So that the first day does not fail on the kinematics, every robot starts in **pass-through** mode: if `cmd_vel` arrives and *never* a `wheel_speeds` message, the simulation converts the twist into four wheels itself — the robot drives as if your solution were already correct. Send the first `wheel_speeds` message and the robot latches for ever into the **wheels** mode: from then on only what you compute applies (the mode is in `/sim/robots` and in the HUD). A broken sign is visible immediately — and that is exactly the intention.

3.5. A second drive train: the steering car

The option `-variant` does double duty. Besides the four mecanum motor variants (stock, slow, fast, agile) it selects a *different kinematic world*: **steering** is a car with one driven axle and one steered axle, so its velocity has one degree of freedom instead of three. Its turn rate is not free but follows from the angle the rack took, $\omega = v \tan \delta / L$, and with it the radius it is driving, $R = L / \tan \delta$; with the default wheel base $L = 1.00$ m and the rack end stop at $\delta_{\max} = 32^\circ$ that is $R_{\min} = 1.60$ m, and a tighter corner is not a driving mistake but a car that does not exist. Two consequences are the point of the exercise: a steering car *cannot strafe* — `vy` has no solution here, and the simulator answers **steering robot cannot strafe**, `vy=0.25` dropped once per robot and simply keeps driving — and its odometry integrates the steering angle it *believes*, because there is no steering encoder on the car: with `-set odom.steer_max_scale=1.1` (an end stop 10 % off) the same drive that was exact returns 45° of yaw error and 1.1 m of place error after 20 s, and nothing ever cancels it out. The four wheel speeds stay on the same topic with the same labels; the rear pair rolls, and their mean is exactly v .

```
./lab run --world production --robot car --variant steering \
    --controller student/steering_example.py --headless --seconds 60
    --truth
```

That controller drives one lap of a rounded rectangle — every corner $R_{\min} + 0.15$ m, because the rack needs half a second to reach 30° — and then parks in front of a shelf by LIDAR. Read it before you write your own: it is one idea (point the wheels at the next waypoint) and forty lines.

3.6. The radio link: commands that can get lost

Nothing in the simulator hands a command to a robot directly; every `cmd_vel` and every `wheel_speeds` message reaches it over a radio link, and with `wifi.enabled=true` that link is a model with two ways to fail: distance, and what stands in the straight line to the hall's one access point. The level at the robot is

$$P_{\text{rssi}} = P_{\text{tx}} - 10n \log_{10} \frac{d}{d_0} - N_{\text{walls}} L_{\text{wall}} + P_{\text{shadow}} \quad q = \text{clamp} \left(\frac{P_{\text{rssi}} - P_{\text{floor}}}{P_{\text{good}} - P_{\text{floor}}}, 0, 1 \right)$$

with the defaults $P_{\text{tx}} = -40$ dBm at $d_0 = 1$ m, path exponent $n = 2.4$ (a furnished hall), $L_{\text{wall}} = 12$ dB per crossed wall and $P_{\text{floor}}/P_{\text{good}} = -85/-50$ dBm. The wall count is the same ray test the LIDAR uses on the same rectangles, so the scan and the radio cannot disagree about which shelves exist. What quality is worth: a frame is lost with probability $(1 - q)^2$ and every frame that arrives is late by $t_{\text{wire}}(1 + 2(1 - q))$, so with the default 20 ms a good link costs 20 ms and a link at $q = 0.32$ costs 47 ms and loses one frame in two. Measured in the `production` hall (access point at 1.0, 2.0 m above the loading area), with the slow fade at zero:

Two things to take from the table. *Walls, not metres, kill a link*: one rack between you and the access point at 8 m is worse than open floor at 15 m. And q reaches 0 while the robot is still *inside* the hall: 23% of that hall's free floor is below the level a robot can be driven on, which is why a hall needs more than one access point in reality.

Below `wifi.link_up_q` (0.15) a countdown runs; if the level stays under it for `wifi.link_timeout` (1.5 s of simulation time), the link is *down*: from then on nothing you send arrives, the robot is drawn with an amber outline, and its `mode` on `/sim/robots` says `autonomy`. What it does then is the option `wifi.autonomy` — `stop` holds the place it was left at (what a command watchdog is for), `dead_reckoning` keeps executing the last command that really arrived and drives on into whatever stands there. That choice is a decision about a practice robot and not about a radio, and it is worth five minutes of discussion.

```
./lab sim --world production --config config/demo_wifi.json      # look at
    layer n
./lab run --world production --config config/demo_wifi.json --robot alice \
    --controller student/link_autonomy_example.py --headless --seconds
    60
```

Layer `n` of the window draws the access point, the line to your robot and a quality bar over it; the readout line prints the same numbers as `wifi q 0.42 -72.3 dBm 1 wall lost 4/120 43 ms`, and `<robot>/link` (5 Hz, JSON in a `std_msgs/String`) carries `t`, `quality`, `rssi_dbm`, `ap`, `up`, `dropped`, `latency_ms` — `ap` included so

that a controller can drive back into coverage *before* the failsafe fires instead of being rescued by it. The example controller above does exactly that walk on purpose: it drives the hall's aisles away from the access point, brakes at $q \leq 0.35$ and prints where the link died (after ≈ 31 s among the racks, with a few hundred frames lost). `config/demo_wifi.json` explains every number in the block, the four-step exercise and what this model deliberately does *not* do: no second access point, no roaming, no shared airtime, no multipath and no retransmission — a frame lost here is lost, and the sequence is never shuffled.

4. Getting started

4.1. Installation

```
sudo apt install python3-pygame          # Ubuntu, enough for the stub mode
git clone <repo>                          # follows in the next step of the
    lab
cd mecanum-lab
./lab run --world maze --robot alice      # sim + your node, ONE process, no
    ROS
```

With ROS 2 (tested with Kilted/Jazzy, Ubuntu 24.04):

```
source /opt/ros/$ROS_DISTRO/setup.bash
./lab sim --world production --robots alice # sim alone, publishes ROS
    topics
```

4.2. Trying it out

```
./lab teleop --robot alice    # the keyboard is the remote: w/s drive, a/d
    turn the chassis
                                # like a car, the arrows drive and strafe like
    a mecanum base
./lab robots                  # who drives, in which mode?
ros2 topic list
ros2 topic echo /alice/odom --once
ros2 topic hz    /alice/scan    # ~20 Hz
ros2 topic echo /alice/wheel_speeds # your four wheel speeds
```

Your node and the keyboard share one topic. `./lab run -controller mein.py` publishes your frames every tick, the keys publish only while one is held: the node drives, the keys interrupt it, and `cmd_timeout` is what ends both. The readout line says which of the two was the last command — `cmd topic 0.04 s, cmd keys 0.02 s, cmd none` — so you never have to wonder which of them a screenshot shows.

One thing more to try, and the only one that needs a sensor you cannot see: the open hall is 30×20 m of floor with a border wall and *one radiation source* planted at (12, 6) m, reaching 4 m (`config/demo_poi_exploration.json`). The LIDAR still sees nothing there — that is what the empty hall is for — so the counter on `<robot>/poi` (intensity 0...1 as JSON in a `std_msgs/String` at 5 Hz, *no* distance) is the only sign

that something is over there. `student/poi_seek_example.py` walks that field with nothing else: straight while the mean over the last second rises, an arc when it stops rising, back to the loudest spot it remembers when the reading falls silent.

```
./lab run --world open --config config/demo_poi_exploration.json \
        --controller student/poi_seek_example.py \
        --seconds 100 --log poi.csv
```

Three runs of that command ended the 100 s of simulation time at a window mean of 0.62, 0.73 and 0.79, arcing near the source; the spread is the node's own loop being paced by the host, which is exactly why that file compares means over a second instead of two single readings. `poi.csv` holds the same series in `intensity_poi` and `name_poi` — `python3 tools/kfplot.py poi.csv -list` prints every column of it. Layer `p` of the window draws the source itself, and `-set poi.publish_distance=true` puts the true metres into the message: switch it on to check an answer afterwards, not to find one.

4.3. Creating a robot through a service

One robot per participant, name unique (`[a-z][a-z0-9_]{1,23}`). **ROS 2 caution:** unlike in ROS 1 there is *no* built-in `std_srvs/SetString`. The simulator therefore offers two ways that work the same for the caller:

1. **Own interface** (recommended, a one-off exercise in `.srv` definitions):

```
# interfaces/mecanum_lab_interfaces/srv/SpawnRobot.srv
string name          # unique
string variant       # "" = automatic (stockslowfastagile)--bool
    successstring messageuint8 indexstring colorstring markerfloat64
    xfloat64 yfloat64 theta

colcon build --packages-select mecanum_lab_interfaces
source install/setup.bash
ros2 service call /sim/spawn_robot mecanum_lab_interfaces/srv/SpawnRobot
\
    "{name: alice, variant: ''}"
```

2. **JSON fallback without building an interface:** handshake over topics, `/sim/spawn_robot/request` (`std_msgs/String`, JSON) → reply on `/sim/spawn_robot/result`, timeout 2 s.

Both paths are wrapped by `./lab spawn -name alice` and `./lab despawn -name alice`; the grader uses this path as well.

4.4. Pitfalls

- **No DISPLAY** (SSH, CI): `SDL_VIDEODRIVER=dummy` or `./lab sim -headless`.
- **ROS not sourced:** `ros2` missing → everything runs automatically over the stub bus; `ros2 topic list` of course shows nothing then.
- **use_sim_time:** a node using wall clock time instead of `/clock` compares timestamps wrongly → `-ros-args -p use_sim_time:=true`.

- **Name already taken:** `success=false`, `message="name already taken"` — first `./lab despawn`, then `spawn` again.
- **Namespace:** topics hang off the robot name, `/alice/odom` $\neq$ `/bob/odom`. Whoever does not pass a robot name waits into the void.
- **Silence:** without a message within 0.35 s the wheels coast down — your node must send at ≥ 10 Hz.

5. Tasks

All thresholds stand in `config/tasks.json` and can be changed there; the grader measures only over topics. Work runs in the production world, `./lab grade -robot alice -task alle` (points per task in table 5).

T1 — Inverse kinematics (30 points)

In `student/controller_template.py` you fill in exactly one function. The rest of the node (topics, rate, watchdog) is already prepared:

```
def inverse_kinematics(vx, vy, omega):
    """[VL, VR, HL, HR] in rad/s: convention from section 2."""
    a = LX + LY # TODO: lever arm for yaw
    return [(vx - vy - a * omega) / R, # TODO: VL
            (vx + vy + a * omega) / R, # TODO: VR
            (vx + vy - a * omega) / R, # TODO: HL
            (vx - vy + a * omega) / R] # TODO: RR
```

Substeps. (1) Start the stub mode, use the teleop, watch the wheels in the HUD. (2) Check one sign after the other: only v_x , then only v_y , then only ω . (3) Run `./lab grade -task kinematik`. **Learning goal:** understand the signs from the roller geometry, not guess them.

The simulator itself computes (excerpt from `mecanum_lab/physics.py`, your calculation has to arrive at the same values):

```
def inverse_kinematics(g, vx, vy, omega) -> list:
    """Body velocity -> four target speeds [VL, VR, HL, HR] in rad/s."""
    a = g.arm
    return [(vx - vy - a * omega) / g.r, (vx + vy + a * omega) / g.r,
            (vx + vy - a * omega) / g.r, (vx - vy + a * omega) / g.r]

def forward_kinematics(g, wheels) -> tuple:
    """Four wheel speeds -> (vx, vy, omega); the exact inverse of the lines above."""
    fl, fr, rl, rr = wheels
    a = g.arm
    return (g.r / 4 * (fl + fr + rl + rr),
            g.r / 4 * (-fl + fr + rl - rr),
            g.r / (4 * a) * (-fl + fr - rl + rr))
```

T2 — Square path with odometry (30 points)

Extend `mission(rob, task)`: four sides of 1.0 m, each 90° counter-clockwise, feedback over `rob.odom()`. Report `done` once you are back at the start pose. Typical approach: straight-run P control on the cross-axis error, deadband for the turn, intermediate targets stored in the odometry. Also measure your own completion error and comment on where it comes from (drift, inertia, angle representation).

T3 — Drive to the goal, the LIDAR protects you (30 points)

Goal: the world's goal (it sits as JSON in `/sim/world`). Direction and distance from the odometry, safety from `rob.scan()`: lateral clearance > 0.16 m, *no* wall contact (`contacts=0`). Useful building blocks: vocabulary from beam averages (not single beams!), speed reduction with the minimum front distance, emergency stop.

T4 — Bonus: GPS instead of odometry (10 points)

Way back to your own start pose, but controlled on `rob.gps()`. Observe and comment on: dead zone of a P controller, measurement noise as jitter, 5 Hz against 50 Hz odometry.

6. Submission and grading

- You hand in `student/<name>_controller.py` in your branch/folder; robot name = file name = participant short name.
- The autograder is decisive: `./lab grade -robot <name> -task alle -json ergebnis.json`. One run per submission, values from table 5.
- Plus a short README (≤ 20 lines): approach, known problems, what you would do differently.
- Self-test before handing in: `./lab run -robot test -controller student/<name>_controller -headless` and `python3 -m pytest tests -q`.

Required text for the commit message

T2: square path with odometry (1.0 m, 90 degrees CCW)

- `inverse_kinematics` corrected to the convention of section 2 (vy sign right)
- turn feedback over `odom.theta`, deadband 0.03 rad, P gain 1.6
- known: completion error ~0.15 m from drift; GPS (T4) would be the next step

Grading rubric

The autograder awards a task's points *whole*: `./lab grade` checks every limit that `config/tasks.json` names for the task and gives its points only when all of them hold — and 0 when one of them does not, naming the failed one in the report. T1 is the only

task with a split, because its three command phases are graded one by one: $30/3 = 10$ points each. The four autograder rows are $30 + 30 + 30 + 10 = 100$ points.

Allowed aids and independent work

Copying solution parts (also from generated answers of other people or language models) leads to withdrawal from the experiment; you may discuss anything. Allowed aids: the script, this handout, docs/, your own notes. The reference solution `student/solution.py` is in the repo and is released *only after the submission deadline*. The four tasks are to be worked in sequence; T1 is the prerequisite for everything else.

7. Outlook

- **Experiment 2 — Odometry fusion:** combine odometry (50 Hz, drift) with GPS (5 Hz, noisy) into one estimate; the compass is missing on purpose.
- **Experiment 3 — Path following:** loop world, follow-the-corridor with LIDAR, speed limit from the curve radius.
- **Experiment 4 — Multiple robots:** several robots, own namespaces, collisions between robots (for that `sensors.py` has to be extended — exactly the kind of change the small simulator allows).

A. Topic, type and API overview

Message layout (excerpt)

```
# /alice/wheel_speeds : std_msgs/msg/Float64MultiArray
msg.data = [w_FL, w_FR, w_RL, w_RR]          # rad/s, the order is the contract

# /alice/odom : nav_msgs/msg/Odometry
odom.pose.pose.position.{x,y}                # world map, odom frame
odom.pose.pose.orientation.z, .w             # quaternion about z (theta)
odom.twist.twist.linear.{x,y}                # body frame, y to the left
odom.twist.twist.angular.z                   # rad/s, CCW positive

# /alice/scan : sensor_msgs/msg/LaserScan
scan.angle_min = 0.0                         # beam 0 = driving direction
scan.angle_increment = 2*pi/360              # increasing index: CCW
scan.ranges[i]                               # m; no hit = range_max (8.0)

# /alice/gps : geometry_msgs/msg/PoseStamped (map frame)
# /alice/link : std_msgs/msg/String, JSON (only with wifi.enabled):
#   {t, quality, rssi_dbm, ap, up, dropped, latency_ms}
# /sim/world : std_msgs/msg/String, JSON: name, size, goal, spawns, walls
```

Student API (robot_io.py)

```
rob = RobotIO("alice")           # ./lab run --robot alice
rob.cmd_vel()                    # Twist or None
rob.odom(); rob.gps(); rob.scan()
rob.link()                       # radio quality, or None (wifi.enabled)
rob.age("odom")                  # age of the last measurement in s
rob.send_wheels([w1, w2, w3, w4])
rob.publish_cmd_vel(vx, vy, omega)
rob.set_mission_state("done")
rob.task(); rob.world(); rob.robots()
serve(module)                    # runner behind the one line in the
    template
```

Contract with serve: If /sim/task is empty or kinematik, every cmd_vel is translated into wheels by inverse_kinematics. For any other task, mission(rob, task) is called exactly once and the state is reported. On kinematik the robot therefore never drives itself: mission() is not called, /mission_state stays idle, and nothing moves until a cmd_vel arrives — from the keyboard in ./lab run or from the grader. That is the exercise and not a bug: T1 is graded on what your conversion makes of the commands it is given, and the first task whose drive is yours is quadrat.

B. Formula sheet

$$\text{IK: } r \begin{pmatrix} w_{\text{FL}} \\ w_{\text{FR}} \\ w_{\text{RL}} \\ w_{\text{RR}} \end{pmatrix} =$$

(8)

$$\text{FK: } v_x = \frac{r}{4} \sum_i w_i, \quad v_y = \frac{r}{4} (-w_{\text{FL}} + w_{\text{FR}} + w_{\text{RL}} - w_{\text{RR}}), \quad \omega = \frac{r}{4a} (-w_{\text{FL}} + w_{\text{FR}} - w_{\text{RL}} + w_{\text{RR}})$$

(9)

$$\text{Limit: } v_{\text{max}} = r w_{\text{max}} = 0.6 \text{ m/s}, \quad \omega_{\text{max}} = \frac{r w_{\text{max}}}{a} = 2.22 \text{ rad/s} \quad (\text{each for } v_y \text{ resp. pure rotation})$$

(10)

$$\text{Normalize the angle: } \theta \in (-\pi, \pi], \quad \Delta\theta = \text{wrap}(\theta_{\text{goal}} - \theta_{\text{current}})$$

(11)

C. Solutions to the prelab questions

1. All four equal: $w = v_x/r = 0.4/0.05 = 8 \text{ rad/s}$. a plays no role, $\omega = 0$.
2. $w_{\text{FL}} = -v_y/r = -5$, $w_{\text{FR}} = +5$, $w_{\text{RL}} = +5$, $w_{\text{RR}} = -5 \text{ rad/s}$. The wheels have to work *against each other*, because every roller transmits force only along d : the left side of the drivetrain is pushed backwards, the right one forwards, and the free-gliding directions allow the vehicle to move lengthwise. No yaw moment: the lever arms cancel $(-, +, +, -)$, signs from eqs. (2) to (5)).

3. $w = \mp a\omega/r = \mp 0.27 \cdot 1.2/0.05 = \mp 6.48 \text{ rad/s}$, concretely $[-6.48, +6.48, -6.48, +6.48]$. FL and RL have the same sign because, in a pure turn, the sign depends only on x_i (wheel pair front/rear) and the direction of rotation — not on left/right; the sideways components are zero here.
4. $w_{\text{FL}} = w_{\text{RR}} = (0.3 - 0.3)/0.05 = 0$, $w_{\text{FR}} = w_{\text{RL}} = 12 \text{ rad/s}$. Standstill at FL/RR is correct. With $w_{\text{max}} = 12$ the diagonal is exactly at the limit: $(v_x - v_y) = 0$, $(v_x + v_y) = 0.6 = r w_{\text{max}}$ — diagonally it ends at $v_x = v_y = 0.3 \text{ m/s}$, although lengthwise 0.6 would be possible.
5. $v_x = \frac{0.05}{4}(8 + 0 + 0 + 8) = 0.2 \text{ m/s}$, $v_y = \frac{0.05}{4}(-8 + 0 + 0 - 8) = -0.2 \text{ m/s}$ (to the right!), $\omega = \frac{0.05}{4 \cdot 0.27}(-8 + 0 - 0 + 8) = 0$. Magnitude $\sqrt{2} \cdot 0.2 = 0.283 \text{ m/s}$, direction 45° front-right. As a commanded twist that would be $(v_x, v_y) = (0.2, -0.2)$.

Table 1: ROS interface (path <robot> = robot name).

Topic	Type (ROS 2)	Meaning
<robot>/cmd_vel	geometry_msgs/msg/Twist	commanded body velocity; $v_y > 0$ means <i>left</i>
<robot>/wheel_speeds	std_msgs/msg/Float64MultiArray (4)	array of four wheel speeds in rad/s — the exercise part
<robot>/odom	nav_msgs/msg/Odometry	dead reckoning from <i>measured</i> wheel speeds, noisy
<robot>/scan	sensor_msgs/msg/LaserScan	360 beams over 2π ; beam 0 points in the driving direction, increasing index runs counter- clockwise; no hit = range_max
<robot>/gps	geometry_msgs/msg/PoseStamped	global position (UWB/MoCap character), noisy
<robot>/truth	geometry_msgs/msg/PoseStamped	exact pose — only if debug_truth is switched on
<robot>/mission_state	std_msgs/msg/String	idle, running, done, failed:<reason>
<robot>/link	std_msgs/msg/String (JSON)	quality of the radio link, only with wifi.enabled — see sec- tion 3.6
/sim/robots	std_msgs/msg/String (JSON)	all robots with color, marker, motor variant, mode
/sim/world	std_msgs/msg/String (JSON)	world name, size, goal, start poses (the same fields ‘doc- s/CONTRACT.md’ §6.2 docu- ments)
/sim/task	std_msgs/msg/String	active task of the grader
/sim/spawn_robot	service, see section 4.3	add a robot (unique name re- quired)
/sim/despawn_robot	service, see section 4.3	remove a robot
/sim/reset	std_srvs/srv/Trigger	put all robots back to the start
/clock	roscpp_msgs/msg/Clock	simulation time (use_sim_time)

Table 2: Parameters from `config/default.json` (excerpt).

Quantity	Value	Remark
Physics rate	50 Hz	fixed step size, so that runs are reproducible
Odometry	50 Hz	noise $\sigma_{\text{wheel}} = 0.04 \text{ rad/s}$ on every wheel measurement $\Rightarrow$ integrated drift; $\sigma_{xy} = 1.5 \text{ mm}$, $\sigma_{\theta} = 1.2 \text{ mrad}$ on the output
LIDAR	20 Hz	360 beams, $0 \dots 8 \text{ m}$, $\sigma = 15 \text{ mm}$; robots do not see each other in experiment 1
Global position	5 Hz	$\sigma_{xy} = 6 \text{ cm}$, $\sigma_{\theta} = 0.03 \text{ rad}$
Motor	$\tau = 0.06 \text{ s}$	first-order inertia, $ \dot{w} \leq 40 \text{ rad/s}^2$, $ w \leq 12 \text{ rad/s}$
Command watchdog	0.35 s	without a new message the wheels coast down
Collision	circle $r = 0.21 \text{ m}$	position is pushed out of the wall, contacts are counted (<code>contacts</code>)

Table 3: Level and quality of the shipped link at spots of the `production` hall (`config/demo_wifi.json`); k is how many racks the line to the access point crosses.

	$k = 0$	$k = 1$	$k = 2$
2 m	-47.2 dBm, $q = 1.00$	-59.2, $q = 0.74$	-71.2, $q = 0.39$
8 m	-61.7, $q = 0.67$	-73.7, $q = 0.32$	-85.7, $q = 0.00$
15 m	-68.2, $q = 0.48$	-80.2, $q = 0.14$	-92.2, $q = 0.00$

driving (teleop)		layers in the window	
w / s	forward, backward	l	lidar scan
a / d	turn left, right	t	odometry trail
q / e	turn left / right	g	GPS fix
up / down	forward, backward	k	estimate + σ ellipse
left / right	strafe sideways	r	wheels and rollers
space	pause	v	velocity vector
esc	quit	c	floor markings
m	all layers at once	z	goal
f	whole world	x	GPS shadow zones
1..9	follow one (0: all)	o	odometry ghost + drift
0	follow all	p	radiation source + field
		h	readout lines
= / -	zoom	n	radio link + access point

Table 4: The keys of the lab window, for teleop and for the keyboard of `./lab run`. The window starts with the raw measurements switched off: what is drawn by default is the robot, its wheels, the odometry estimate and the readout lines, and everything a sensor measures has to be asked for with its key. That is on purpose — the estimate and the measurement look the same when both are on, and telling them apart is the exercise. `m` switches every layer on at once.

Table 5: Tasks, points, success conditions.

ID	Content	Limit	Points
kinematikT1	<code>cmd_vel</code> $\rightarrow$ four wheels. Three phases of 3 s each: $v_x = 0.3$; $v_y = 0.3$; $\omega = 0.6$.	$\Delta x > 0.35$ m forward $\Delta y > 0.35$ m left $\Delta \theta > 1.2$ rad turning cross-axis error < 0.15 m straight, turning < 0.35 m	30
quadrat	T2: square path, four sides of 1.0 m, 90° CCW, using only odometry.	completion error < 0.25 m, $\Delta \theta < 20^\circ$, path length 3.0...12.0 m, 0 contacts, under 90 s	30
korridor	T3: drive to the world's goal, LIDAR prevents every wall contact.	target error < 0.30 m, 0 contacts, lateral clearance > 0.16 m on straight runs ($ \omega < 0.25$), under 120 s	30
gps_anfahrtT4	(bonus): the loading spot — the <i>last spawn pose</i> of this world, deliberately <i>not</i> your own start pose, so that having stood there already is not a solution. Guided by GPS, not by odometry.	error < 0.45 m, 0 contacts, under 90 s	10

Criterion	Points	checked by
T1 kinematics (three phases of 10 P each)	30	autograder
T2 square path (every limit of the task at once, no partial credit)	30 or 0	autograder
T3 goal without contact (every limit of the task at once, no partial credit)	30 or 0	autograder
T4 bonus GPS (the same rule)	10 or 0	autograder
short summary in the README	0–5	supervision